Manufacturing Downtime

Data Analyst Specialist

CAI3-DAT1-G2

By

Ahmed Taher Abdelhamed Youssef

Abdelrhman Mohammed Mohammed Fekry

Michael Emil Abdou Habib

Aya Ramadan Mahmoud Rashed

Supervised by

Eng. Ahmed Samir Ahmed

1 Introduction

1.1 Project Title

Manufacturing Downtime

1.2 Background

Manufacturing industries operate in environments where efficiency and reliability are critical. Even short periods of downtime can significantly disrupt production schedules, delay deliveries, and increase operational costs. Downtime may be caused by equipment failures, operator errors, supply chain delays, or planned maintenance, but without proper tracking and analysis, it is often difficult to identify the root causes. In competitive markets, minimizing downtime is directly linked to higher productivity, better resource utilization, and improved profitability.

1.3 Problem Statement

Manufacturing downtime disrupts productivity and raises hidden costs, yet many organizations struggle to understand its causes due to poor or fragmented data. Without proper analysis, companies often remain reactive, relying only on descriptive statistics instead of predictive insights. This lack of foresight prevents proactive planning and results in wasted labor, reduced output, and customer dissatisfaction. The project aims to fill this gap by systematically cleaning, modeling, and analyzing downtime data, creating a foundation for forecasting and strategic decisions.

1.4 Project Objectives

1. Analyze downtime patterns and causes

The first objective is to explore and understand the main factors contributing to production downtime. This involves analyzing patterns such as which products, shifts, or operators experience the most downtime and identifying recurring causes like machine failures, operator errors, or batch changes. By visualizing and quantifying these trends, the analysis aims to highlight inefficiencies, prioritize problem areas, and support decision-makers in improving line productivity.

2. Build predictive models to forecast downtime

The second objective is to apply statistical and machine learning methods to forecast future downtime events. Using historical data, the model should predict the expected downtime for the next day of operation and estimate its impact on production capacity. This predictive component allows proactive planning — helping managers schedule maintenance, allocate resources efficiently, and reduce the likelihood of unexpected production interruptions.

3. Present insights through interactive dashboards

The final objective is to communicate findings clearly through interactive dashboards built in Tableau or Power BI. These dashboards will display key performance indicators (KPIs), downtime trends, and forecasting results in an accessible, visual format. By integrating filters and slicers, stakeholders can explore the data dynamically — for example, by product, operator, or downtime factor — making it easier to derive actionable insights and support data-driven decision-making.

1.5 Project Scope

The project centers on analyzing manufacturing downtime using a provided dataset, progressing through several analytical phases, including data cleaning, modeling, forecasting, and visualization. The defined scope covers activities such as importing and preprocessing data, constructing relational data structures, formulating analytical and forecasting questions, and deriving insights through reports and interactive dashboards. The work is limited to the offline dataset supplied, with the objective of uncovering meaningful patterns and predictive trends that enhance operational efficiency and support informed decision-making.

1.6 Expected Outcomes

By the end of the project, several tangible outcomes are expected. A fully cleaned and preprocessed dataset will be produced, ensuring consistency, completeness, and readiness for analysis. The analysis phase will generate insights about downtime causes, operator performance, and production efficiency trends. Forecasting models are expected to estimate downtime duration or likelihood, supporting proactive decision-making. Finally, the results will be communicated through interactive dashboards and a structured report, providing clear visual summaries and data-driven recommendations for process improvement.

1.7 Tools used

Excel, SQL, and Python were used for data exploration, cleaning, preprocessing, and building the data model. Tableau and Power BI were later utilized to visualize key performance indicators, downtime trends, and forecasting insights through interactive dashboards.

1.8 Deliverables

The main deliveries of the project include:

Cleaned Dataset: A refined, well-structured dataset free from duplicates and inconsistencies.

Analysis and Forecasting Reports: Documentation summarizing analytical insights and predictive model results.

Visualization Dashboard: An interactive Tableau and Power BI dashboard showcasing downtime metrics and trends.

Final Presentation: A professional presentation summarizing objectives, methodology, analysis, forecasting, and insights.

2 Dataset

2.1 Dataset source and description

.xlsx and .csv files provided under the project titled: *Manufacturing Downtime*.

2.2 Dataset structure

Four Excel worksheets were provided, each containing specific information relevant to the manufacturing process, as outlined below:

- **Line Productivity:** Batch details: Date, Product, Operator, Start/End Time.

Date	Product	Batch	Operator	Start Time	End Time
2024-08-29	OR-600	422111	Mac	11:50:00	14:05:00
2024-08-29	LE-600	422112	Mac	14:05:00	15:45:00
2024-08-29	LE-600	422113	Mac	15:45:00	17:35:00
2024-08-29	LE-600	422114	Mac	17:35:00	19:15:00
2024-08-29	LE-600	422115	Charlie	19:15:00	20:39:00
2024-08-29	LE-600	422116	Charlie	20:39:00	21:39:00
2024-08-29	LE-600	422117	Charlie	21:39:00	22:54:00

Figure 2-1 Line Productivity Excel Worksheet

- **Products:** Product like Flavor, Size, Min batch time.

Product	Flavor	Size	Min batch time
OR-600	Orange	600 ml	60
LE-600	Lemon lime	600 ml	60
CO-600	Cola	600 ml	60
DC-600	Diet Cola	600 ml	60
RB-600	Root Berry	600 ml	60
CO-2L	Cola	2 L	98

Figure 2-2 Products Excel Worksheet

- **Downtime Factors:** Factor ID, Description, Operator Error.

Factor	Description	Operator Error
1	Emergency stop	No
2	Batch change	Yes
3	Labeling error	No
4	Inventory shortage	No
5	Product spill	Yes

Figure 2-3 Downtime Factors Excel Worksheet

- **Line Downtime:** Batch – Factor mapping with delay duration.

	Downtime factor											
Batch	1	2	3	4	5	6	7	8	9	10	11	12
422111		60					15					
422112		20						20				
422113		50										
422114				25		15						
422115										24		
422116												
422117		10				5						

Figure 2-4 Line Downtime Excel Worksheet

2.3 Data Dictionary

Provide a table summarizing columns, types, and meanings.

Table	Field	Description
Line productivity		Fact table containing details for each batch produced
Line productivity	Date	Date the batch was produced
Line productivity	Product	ID for the product produced in the batch
Line productivity	Batch	Unique ID for the batch produced
Line productivity	Operator	Production line operator in charge of the batch
Line productivity	Start Time	Time the batch production started
Line productivity	End Time	Time the batch production ended
Products		Dimension table with details on each product
Products	Product	Unique product ID
Products	Flavor	Soda flavor for the product
Products	Size	Product size (volume)
Products	Min batch time	Minimum time required to produce a batch (with no downtime)
Line downtime		Fact table containing downtime (in minutes) by factor for each batch
Line downtime	Batch	Unique ID for the batch produced
Line downtime	Downtime factor	Downtime minutes for each factor ID (across columns)
Downtime factors		Dimension table with details on each downtime factor
Downtime factors	Factor	Unique ID for each downtime factor
Downtime factors	Description	Downtime factor description
Downtime factors	Operator Error	Is this due to operator error? (Yes/No)

Figure 2-5 Data Dictionary

3 Phase ONE: Data Model, Cleaning & Preprocessing

3.1 Data Cleaning & Preprocessing

3.1.1 Data Transformation

Data transformation makes raw data usable. It fixes inconsistent formats, units, and column names, ensuring the dataset is clean, structured, and ready for accurate analysis.

What was done?

- Converted Time Formats

The Start Time and End Time columns were standardized to the proper *Time* datatype. This ensured accurate duration calculations and eliminated inconsistencies caused by mixed or text-based time formats.

- Unified Product Variants

Product labels were cleaned and corrected to distinguish between similar items, such as separating **Cola (600 ml)** and **Cola (2L)** into two properly defined product categories. This prevented incorrect aggregations and improved product-level analysis.

- Standardized Units

All size values were converted into a consistent numeric format to ensure comparability. For example, *2 L* was changed to *2000 ml*, and *600 ml* was standardized as *600 ml*. This unification ensured accurate filtering, grouping, and calculations.

- Renamed Columns for Clarity

Columns were renamed to make the dataset clearer and more descriptive. The *Size* column became *Size_ml*, and *Min Batch Time* was updated to *Min batch time (mins)*. These changes improved readability and reduced ambiguity during analysis.

3.1.1.1 Microsoft Excel

All data transformations in Excel were performed using **Power Query**, which provided a structured and repeatable workflow. Through Power Query, time columns were converted to proper formats, product labels were cleaned, units were standardized, and column names were unified for clarity. Power Query allowed these changes to be applied efficiently and consistently across the dataset, ensuring a clean and reliable foundation for further analysis.

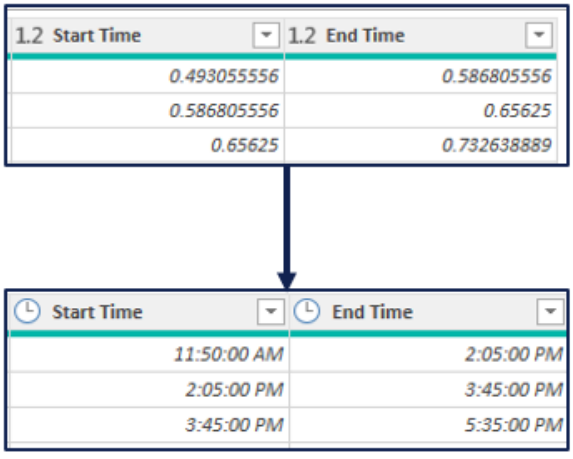


Figure 3-1 Data Transformation using Power Query

3.1.1.2 Structured Query Language (SQL)

The dataset, titled "Manufacturing Downtime", was initially provided in Excel (.xlsx) format, along with a supporting data dictionary in .csv format that described the meaning of each field.

To enable structured querying and analysis, the data was imported into SQL Server by first exporting each worksheet to a separate .csv file, then conducting the "Bulk Insert" command to create the corresponding tables: Line_Productivity, Products, Downtime_Factors, and Line_Downtime. However, only for **Line Downtime** worksheet, unpivoting was required before importing to ensure each factor and its corresponding downtime delay were represented as separate rows. Once transformed, the data was imported into SQL and assigned to their respective tables, which had been created in advance with explicitly defined data types for each column. Primary keys (PKs) were also specified to maintain data integrity. **Error! Reference source not found.**, **Error! Reference source not found.**, **Error! Reference source not found.**, and **Error! Reference source not found.** below show the queries of creating each table, then performing the "Bulk Insert" command on each one.

```
create table Line_Productivity (
Date Date,
Product varchar(20),
Batch varchar(20),
Operator varchar(20),
Start_Time time,
End_Time time,
Constraint Batch_Pk  primary key (Batch)
)

Bulk insert dbo.Line_Productivity
from 'C:\Users\Ahmed Taher\Desktop\final edits 6-11-25\Manufacturing_Line_Productivity.csv'
with (
format='CSV',
firstrow=2
)
```

Figure 3-2 Line Productivity table creation and bulk insert

```
create table Products (
Product varchar(20),
Flavor varchar(20),
Size varchar(20),
Min_Batch_Time_mins int,

constraint Product_Pk  primary key (Product)
)
Bulk insert dbo.Products
from 'C:\Users\Ahmed Taher\Desktop\DEPI ASSIGNMENTS\GRADUATION PROJECT\Datasets\Datasets\Manufacturing Downtime\Products.csv'
with (
format='CSV',
firstrow=2
)
```

Figure 3-3 Products table creation and bulk insert

```
create table Downtime_Factors (
Factor varchar(5),
Description varchar(30),
Operator_Error varchar(5),

Constraint Factor_Pk  primary key (Factor)
)

Bulk insert dbo.Downtime_Factors
from 'C:\Users\Ahmed Taher\Desktop\DEPI ASSIGNMENTS\GRADUATION PROJECT\Datasets\Datasets\Manufacturing Downtime\Downtime factors.csv'
with (
format='CSV',
firstrow=2
)
```

Figure 3-4 Downtime Factors table creation and bulk insert

```
create table Line_Downtime (
Batch varchar(20),
Factor varchar(5),
Downtime_in_mins int ,
Description varchar(25),
Operator_Error varchar(5)

Constraint Batch_Factor_Pk primary key (Batch, Factor)
)

Bulk insert dbo.Line_Downtime
from 'C:\Users\Ahmed Taher\Desktop\final edits 6-11-25\Line Downtime.csv'
with (
format='CSV',
firstrow=2
)
```

Figure 3-5 Line Downtime table creation and bulk insert

After importing the dataset, data transformation steps were applied using SQL as follows, Figure 3-6 showing data transformation using SQL query.

```sql
-- Separate Cola 600 ml and Cola 2L into clear distinct products
UPDATE p
SET Product =
    CASE
        WHEN Product = 'Cola' AND Size LIKE '%600%' THEN 'Cola (600 ml)'
        WHEN Product = 'Cola' AND (Size LIKE '%2L%' OR Size LIKE '%2 L%') THEN 'Cola (2L)'
        ELSE Product
    END
FROM Products AS p;
```

```sql
-- Standardized Units
-- Convert all size values into numeric milliliters
UPDATE Products
SET Size =
    CASE
        WHEN Size LIKE '%2L%' OR Size LIKE '%2 L%' THEN '2000'
        WHEN Size LIKE '%600%' THEN '600'
        ELSE Size
    END;
```

Figure 3-6 Data Transformation using SQL

3.1.1.3 Python

The dataset was provided in .xlsx format, accompanied by a dictionary file in .csv format that clarifies the meaning of each field. Under the main title "Manufacturing Downtime", the Excel file contained four worksheets, which were imported into Python (Jupyber notebook) using the pandas library. These sheets are: **(A)** *Line Productivity*, **(B)** *Products*, **(C)** *Downtime Factors*, and **(D)** *Line Downtime*. Each sheet was loaded individually using the "pd.read_excel()" function, specifying the sheet name parameter to ensure proper data separation and handling. This approach allows direct access to each table for cleaning, transformation, and analysis in subsequent steps. **Error! Reference source not found.** shows the code of importing the Excel file into Python (Jupyter notebook).

```
import pandas as pd
```

```
df=pd.read_excel('C:/Users/Ahmed Taher/Desktop/DEPI ASSIGNMENTS/GRADUATION PROJECT/Datasets/Datasets/Manufacturing Downtime/Manufacturing_Line_Productiv
df
sheets=pd.ExcelFile('C:/Users/Ahmed Taher/Desktop/DEPI ASSIGNMENTS/GRADUATION PROJECT/Datasets/Datasets/Manufacturing Downtime/Manufacturing_Line_Produc
sheets
```

```
['Line productivity', 'Products', 'Downtime factors', 'Line downtime']
```

```
Line_productivity= pd.read_excel('F:/DEPI/Datasets/Manufacturing Downtime/Manufacturing_Line_Productivity.xlsx',sheet_name='Line productivity')
Products= pd.read_excel('F:/DEPI/Datasets/Manufacturing Downtime/Manufacturing_Line_Productivity.xlsx',sheet_name='Products')
Downtime_factors= pd.read_excel('F:/DEPI/Datasets/Manufacturing Downtime/Manufacturing_Line_Productivity.xlsx',sheet_name='Downtime factors')
Line_downtime= pd.read_excel('F:/DEPI/Datasets/Manufacturing Downtime/Manufacturing_Line_Productivity.xlsx',sheet_name='Line downtime',header=1)
```

Figure 3-7 Importing data to Python

After importing the dataset, data transformation was then done using Python (Jupyter). Figure 3-8 showing data transformation syntax using Pytohn.

```
import pandas as pd
```

```
start = pd.to_datetime(Line_productivity["Start Time"], format="%H:%M:%S")
end   = pd.to_datetime(Line_productivity["End Time"], format="%H:%M:%S")
end = end.where(end >= start, end + pd.Timedelta(days=1))
Line_productivity["Actual time"] = (end - start).dt.total_seconds() / 60
```

```
Products["Size"] = (
    Products["Size"]
    .str.replace("ml", "", regex=False)
    .str.replace("L", "000", regex=False)
    .str.replace(" ", "", regex=False)
    .astype(int)
)
```

Figure 3-8 Data Transformation using Python

3.1.2 Handling missing values

Missing values can break calculations, disrupt time-based metrics, and create misleading insights. Addressing them ensures the dataset is complete, consistent, and analytically sound. By verifying that no fields are left blank, especially in critical columns such as durations, factors, and timestamps, wecan then guarantee that every KPI and summary truly represents real production performance, without gaps that could distort trends or hide important issues.

3.1.2.1 Microsoft Excel

Using Power Query, missing values check been done, and there was no missing values to be found. Figure 3-9 showing no missing values in the dataset using Power Query

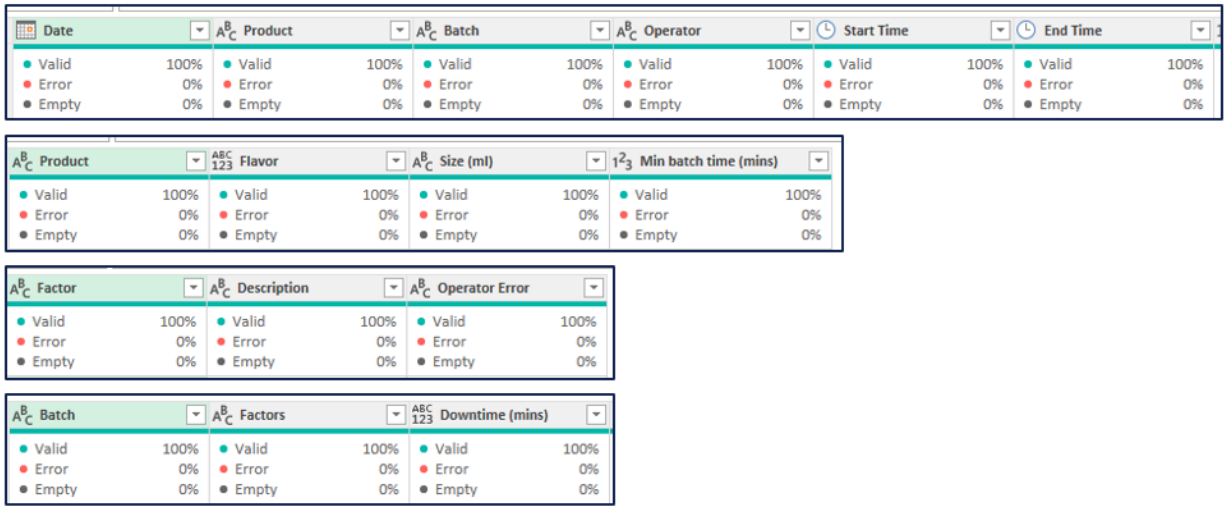


Figure 3-9 Missing values check using Power Query

3.1.2.1 Structured Query Language (SQL)

Using SQL, checking missing values was done for the whole dataset as follows. Figure 3-10 showing missing values check

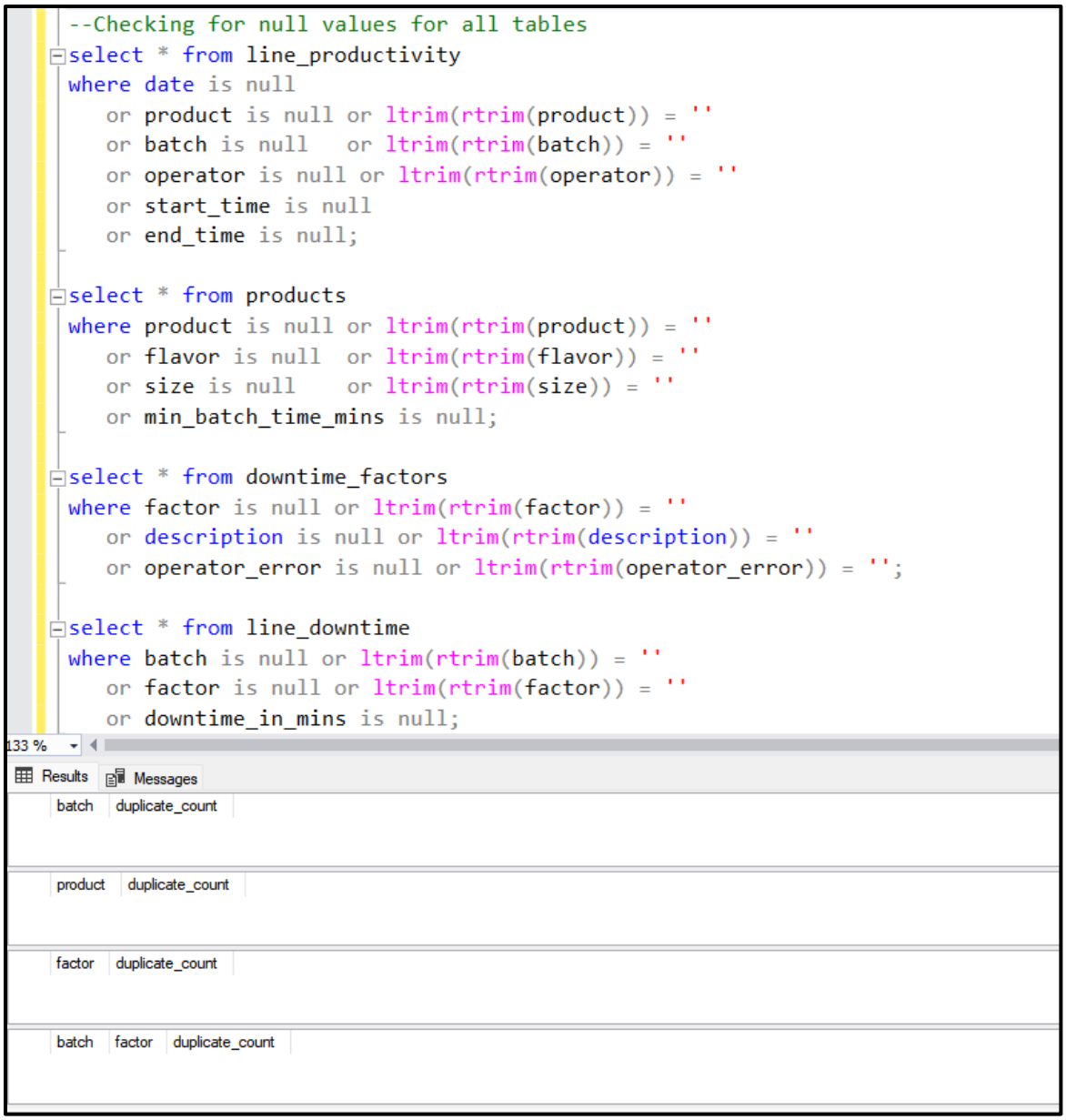


Figure 3-10 checking missing values using SQL

3.1.2.1 Python

Using Python, checking missing values was done for the whole dataset as follows. Figure 3-10 showing missing values check

```
[11]: Line_productivity.isnull().sum()

[11]: Date          0
      Product       0
      Batch         0
      Operator      0
      Start Time    0
      End Time      0
      dtype: int64

[12]: Line_productivity.duplicated().sum()

[12]: np.int64(0)
```

Figure 3-11 Checking for nulls and duplicated values in Line Productivity

```
[21]: Products.duplicated().sum()

[21]: np.int64(0)

[22]: Products.isnull().sum()

[22]: Product           0
      Flavor            0
      Size(ml)          0
      Min batch time    0
      dtype: int64
```

Figure 3-12 Checking for nulls and duplicated values in Products

```
[25]: Downtime_factors.duplicated().sum()

[25]: np.int64(0)

[26]: Downtime_factors.isnull().sum()

[26]: Factor           0
      Description      0
      Operator Error   0
      dtype: int64
```

Figure 3-13 Checking for nulls and duplicated values in Downtime Factors

```
[32]: Line_downtime_long.duplicated().sum()

[32]: np.int64(0)

[33]: Line_downtime_long.isnull().sum()

[33]: Batch              0
      Factor             0
      Downtime_Minutes   0
      dtype: int64
```

Figure 3-14 Checking for nulls and duplicated values in Line Downtime

3.1.3 Checking Duplicates

3.1.3.1 Microsoft Excel

Using Power Query, Duplicated values check been done, and there was no duplicated values to be found. Figure 3-9 showing no missing values in the dataset using Power Query

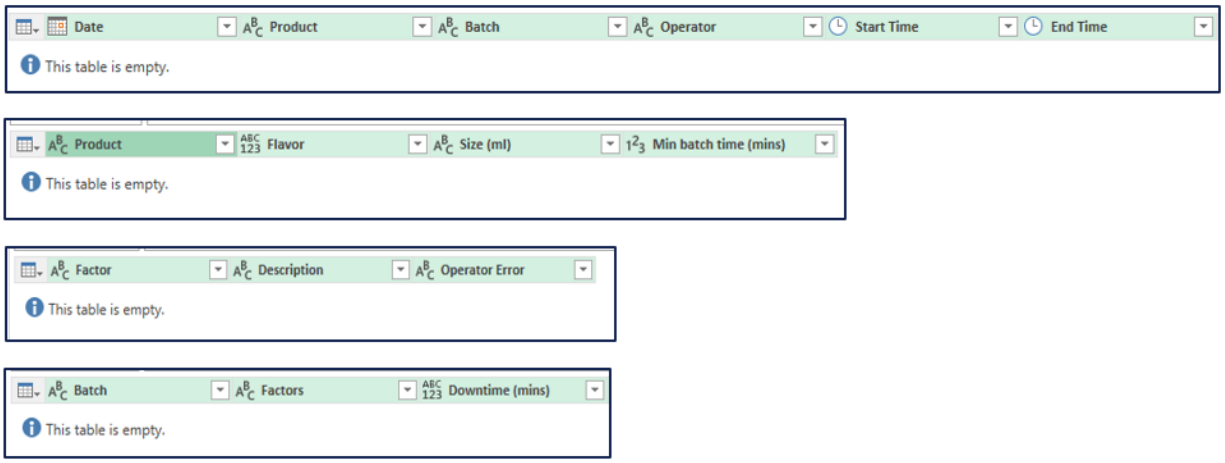


Figure 3-15 Checking Duplicates using Power Query

3.1.3.2 Structured Query Language (SQL)

Using SQL, checking duplicated values was done for the whole dataset as follows. Figure 3-10 showing missing values check

```sql
-- check duplicate values for all tables
select batch, count(*) as duplicate_count
from line_productivity
group by batch
having count(*) > 1;

select product, count(*) as duplicate_count
from products
group by product
having count(*) > 1;

select factor, count(*) as duplicate_count
from downtime_factors
group by factor
having count(*) > 1;

select batch, factor, count(*) as duplicate_count
from line_downtime
group by batch, factor
having count(*) > 1;
```

133 %

Results | Messages

batch	duplicate_count

product	duplicate_count

factor	duplicate_count

batch	factor	duplicate_count

Figure 3-16 Checking duplicates using SQL

3.1.3.3 Python

Using Python, checking duplicated values was done for the whole dataset as follows. Figure 3-10 showing missing values check

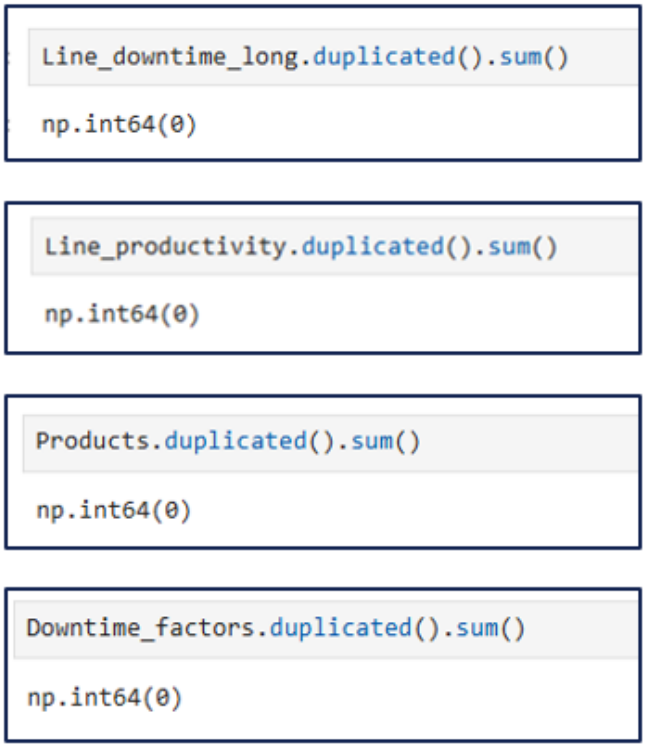


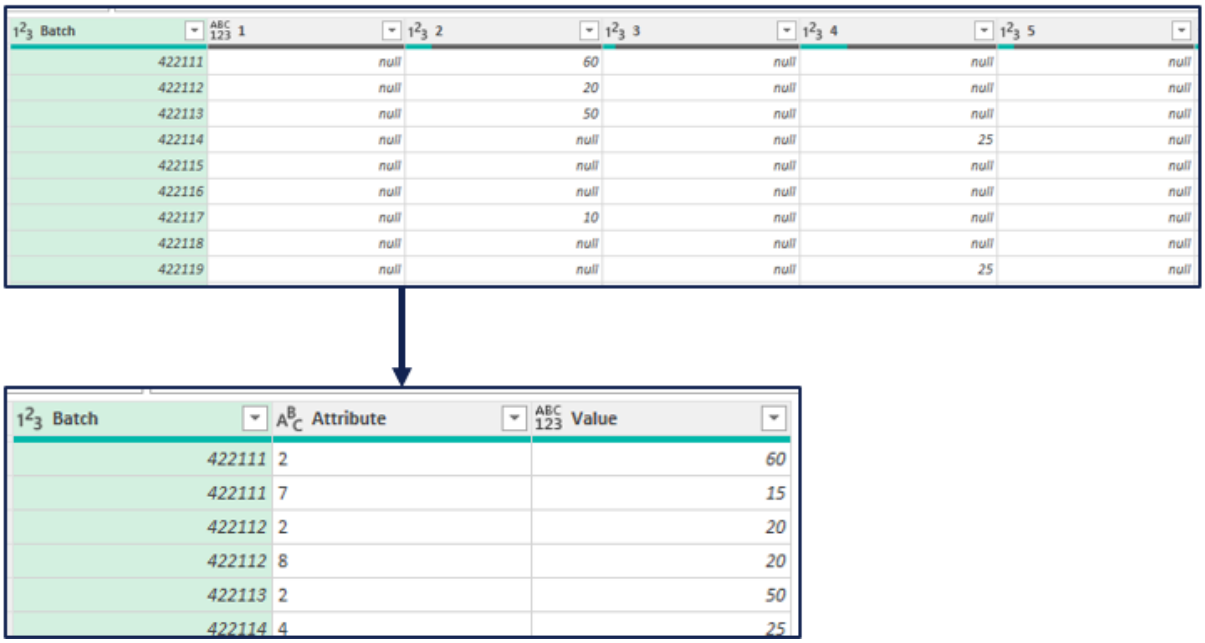
Figure 3-17 Duplicated values check using Python

3.1.4 Unpivoting Columns

Unpivoting transforms the downtime data from a wide format into a long, factor-level structure. This step takes multiple downtime-factor columns and converts them into individual rows, each representing a single factor and its corresponding delay. By doing this, the dataset becomes fully analysis-ready, allowing accurate aggregation, clearer comparisons across factors, and more meaningful visualizations such as Pareto charts and cause-based breakdowns.

3.1.4.1 Microsoft Excel

Using Power Query, Unpivoting columns was done. Fig showing unpivoting columns.



Batch	1	2	3	4	5
422111	null	60	null	null	null
422112	null	20	null	null	null
422113	null	50	null	null	null
422114	null	null	null	25	null
422115	null	null	null	null	null
422116	null	null	null	null	null
422117	null	10	null	null	null
422118	null	null	null	null	null
422119	null	null	null	25	null

Batch	Attribute	Value
422111	2	60
422111	7	15
422112	2	20
422112	8	20
422113	2	50
422114	4	25

Figure 3-18 Unpivoting Columns using Power Query

3.1.4.1 Python

Using Python (melting) syntax, Unpivoting columns was done. fig showing Python syntax

```
Line_downtime_long = (
    Line_downtime.melt(
        id_vars=["Batch"],
        value_vars=list(range(1, 13)),
        var_name="Factor",
        value_name="Downtime_Minutes"
    )
    .dropna(subset=["Downtime_Minutes"])
    .sort_values(["Batch", "Factor"])

    .reset_index(drop=True)
)
```

```
Line_downtime_long
```

Figure 3-19 Unpivoting columns using Python

3.2 Building the Data model

3.2.1 Microsoft Excel

Using **Power Pivot** in Excel, we developed a data model that connected the different tables in the dataset. Relationships were created by linking key columns across tables, ensuring that all tables could interact seamlessly. This allowed preparing the data for analysis without manually merging tables each time. **Error! Reference source not found.** and **Error! Reference source not found.** below outlines which columns were used to establish relationships.

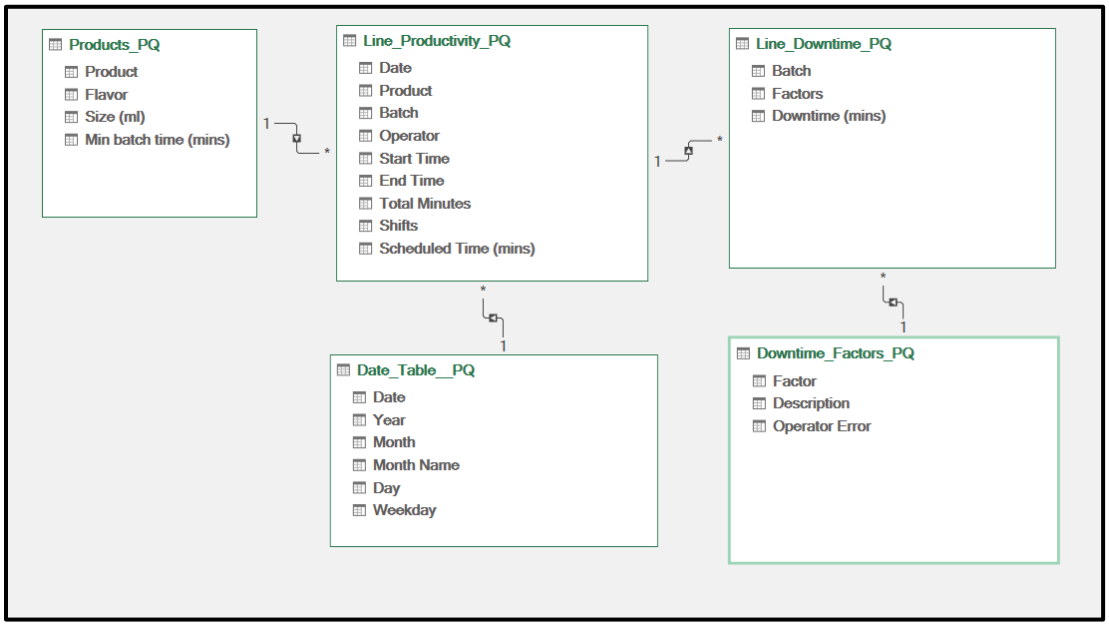


Figure 3-20 Data model

Table 3-1 Data model relations

Table 1 (column)	To	Table 2 (column)
Line Productivity (Product)		Products (Product)
Line Downtime (Batch)		Line Productivity (Batch)
Line Downtime (Factor)		Downtime Factors (Factor)
Line Productivity (Date)		Date (Date)

3.2.2 Structured Query Language (SQL)

The SQL data model was built by establishing relationships between the four tables using primary and foreign keys. This structure ensured that all tables are connected seamlessly. The relationships were defined as follows. Figure 3-21, and Figure 3-22 show the query and schema of the data model. relations shows relationships between tables.

```
--********* Adding Foreign Keys to make relationships *********--

Alter table Line_Productivity
add constraint Product_Fk Foreign Key (Product) references Products(Product)

Alter table Line_Downtime
add   constraint Factor_Fk Foreign Key (Factor) references Downtime_Factors(Factor)

Alter table Line_Downtime
add constraint Batch_Fk  Foreign Key (Batch) references Line_Productivity (Batch)
```

Figure 3-21 Relation between tables through Primary and Foreign keys

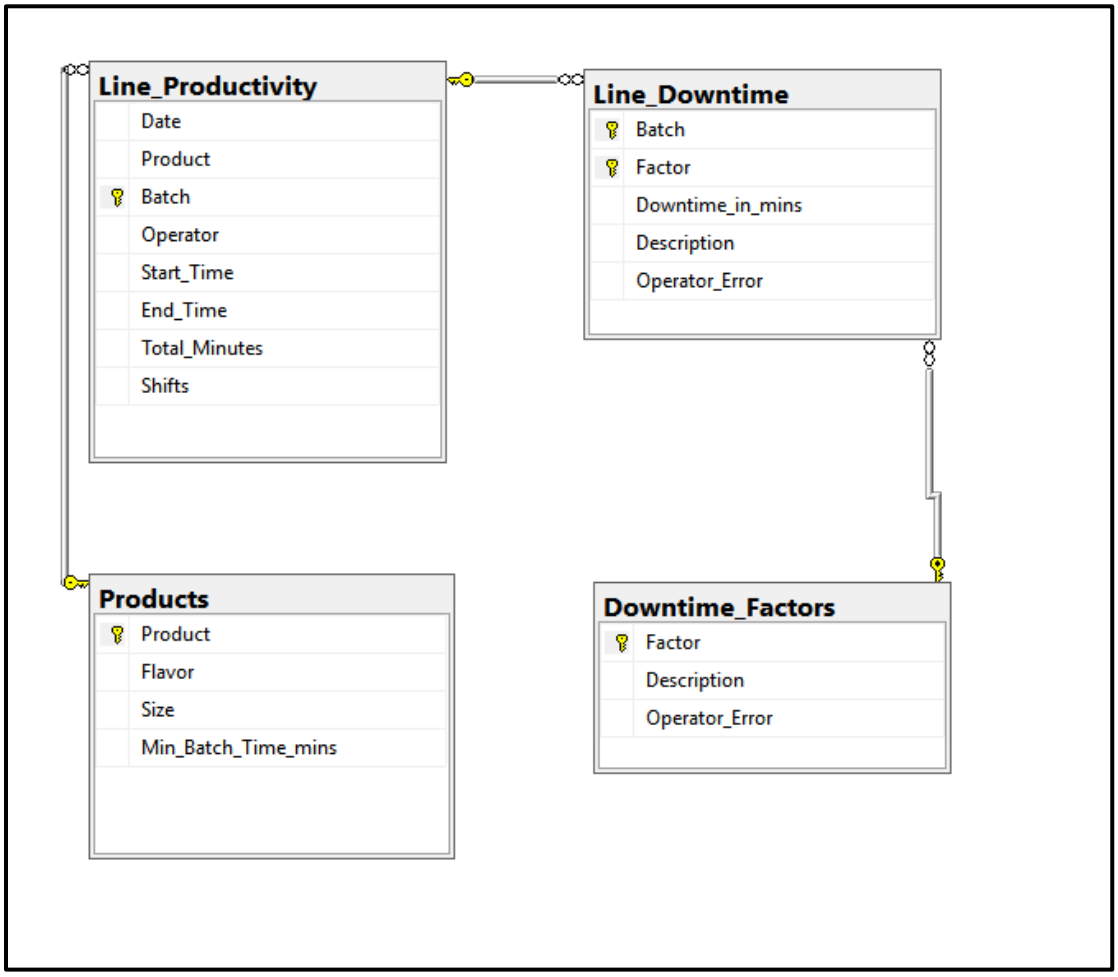


Figure 3-22 Data model schema (Galaxy schema)

3.3 Summary

The preprocessing phase focused on transforming the raw dataset into a clean, structured, and analysis-ready format using three tools: Excel, SQL, and Python. Each tool played a complementary role in standardizing formats, correcting inconsistencies, and restructuring the data to support accurate downstream analysis.

Data transformation included converting time columns to proper formats, unifying product labels, standardizing units, and renaming columns for clarity. These adjustments ensured consistent formatting and eliminated ambiguities across the dataset. Additional transformations such as unpivoting the downtime information allowed each downtime factor to be captured at a granular level, enabling deeper and more accurate analytical insights.

Missing values and duplicates were checked systematically in all three tools. The dataset was validated to be free of missing or duplicate records, ensuring all KPIs, summaries, and visualizations would be reliable. This data-quality verification step helped safeguard the accuracy of operational metrics such as batch duration, downtime totals, and factor-level breakdowns.

In Excel, Power Query was used extensively to automate cleaning workflows and to perform key transformations such as unpivoting and normalization of columns. SQL was used to enforce structure by defining schemas, data types, and primary/foreign key relationships. Python further supported transformation and validation through pandas operations, including melting, renaming, type conversion, and date/time adjustments.

Finally, a data model was constructed in Excel and SQL to formally establish the relationships between tables—linking products, batches, downtime events, and factors. Python maintained these relationships logically through consistent keys across DataFrames. Together, these steps produced a standardized, well-modeled dataset that provided a strong foundation for subsequent analysis, forecasting, and visualization phases.

4 Phase TWO: Analysis Questions

4.1 Identifying Analysis Questions

After cleaning the dataset, the next step was to explore it using a structured set of SMART analysis questions that would reveal meaningful patterns across the entire operation. These questions were organized into five focused analytical areas: **Overview**, **Production Line Performance**, **Operator Performance**, **Product Analysis**, and **Downtime Factors**. Grouping the questions this way ensured comprehensive coverage of all operational dimensions and enabled a clear, systematic understanding of what drives downtime and production efficiency.

Table 4-1 Analysis Questions

No.	Analysis Question	Dashboard
1	What is the total downtime recorded?	**Overview**
2	How many downtime events occurred?	
3	What is the total number of batches?	
4	What is the total scheduled production time? (batch time)	
5	How much time was spent producing?	
6	What is the percentage of total downtime impact?	
7	How does daily production volume change over time?	
8	What is the min batch time, downtime, and total duration for each batch?	**Production Line Performance**
9	What is the average batch duration per product and operator?	
10	Which operator completes batches closest to the Min Batch Time?	
11	Which product variant deviates most from ideal batch time?	
12	Which products have the highest number of batches produced?	
13	Which product consumes the most production time?	
14	What is the total number of downtime factors?	**Downtime Factors**
15	What is the most time-consuming downtime factor?	
16	What is the most occurred downtime factor?	
17	How much cumulative downtime did each factor contribute over time?	
18	Which downtime factors occur most often in each shift?	
19	Which downtime factors occur most often in each day?	
20	Which operator has the highest downtime?	**Operators**
21	How many batches does each operator handle?	
22	How does each operator's efficiency change with their downtime?	
23	Is most of the downtime caused by operators or by non-operator?	
24	Which operators face more operator-related downtime factors?	
25	Which operator completes batches closest to the Min Batch Time?	
26	Which product has the most downtime?	**Products**
27	What is the percentage of downtime impact per product?	
28	Are certain products more affected by specific downtime factors?	

4.2 Analysis

Using the three mentioned tools, analysis of questions was carried out. Figures below showing the analysis for each question using the three tools

What is the total downtime recorded?
Total Downtime
1388

Figure 4-1 Analyze Q1

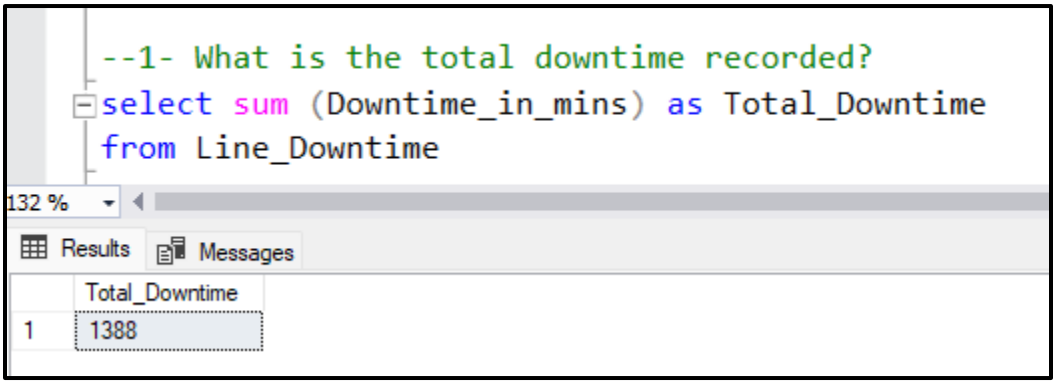


Figure 4-2 Analyze Q1

```
# What is the total downtime recorded?
total_downtime=Line_downtime_long['Downtime_Minutes'].sum()
total_downtime

np.int64(1388)
```

Figure 4-3 Analyze Q1

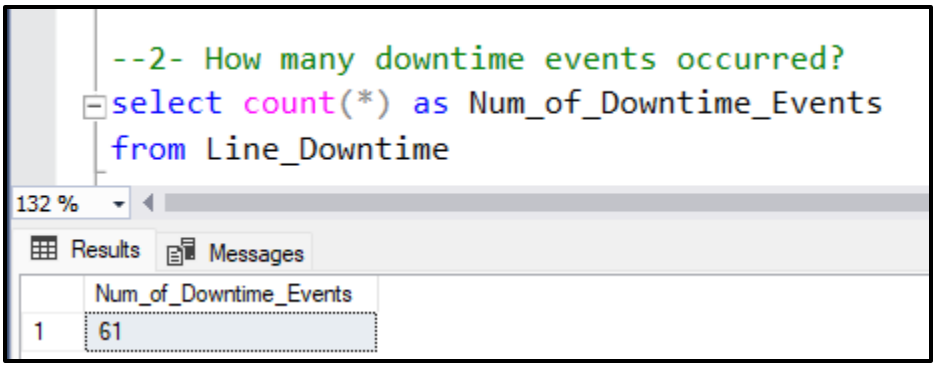


Figure 4-4 Analyze Q2

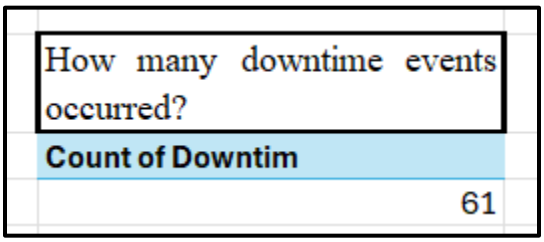


Figure 4-5 Analyze Q2

Figure 4-6 Analyze Q2

What is the total number of batches?

Count of Batch
38

Figure 4-7 Analyze Q3

```
--3- What is the total number of batches?
select count(*) as Total_Batches
from Line_Productivity
```

132 %

Results | Messages

	Total_Batches
1	38

Figure 4-8 Analyze Q3

```
#What is the total number of batches?
count_Batches=Line_productivity['Batch'].count()
count_Batches
```

```
np.int64(38)
```

Figure 4-9 Analyze Q3

What is the total production time?

Total Actual Duration (mins)
3858

Figure 4-10 Analyze Q4

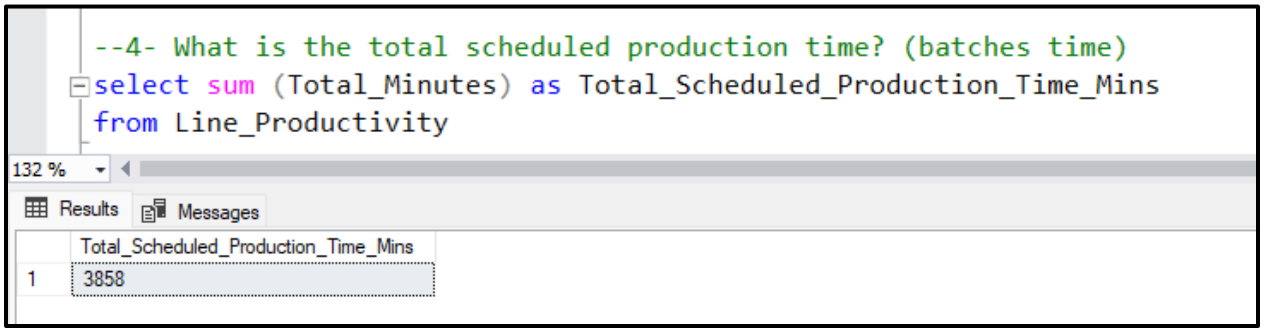


Figure 4-11 Analyze Q4

```
#What is the total scheduled production time? (batches time)
total_scheduled_time = Line_productivity['Actual time'].sum()
total_scheduled_time

np.int64(3858)
```

Figure 4-12 Analyze Q4

How much time was spent producing?

Sum of Scheduled Time (mins)
2470

Figure 4-13 Analyze Q5

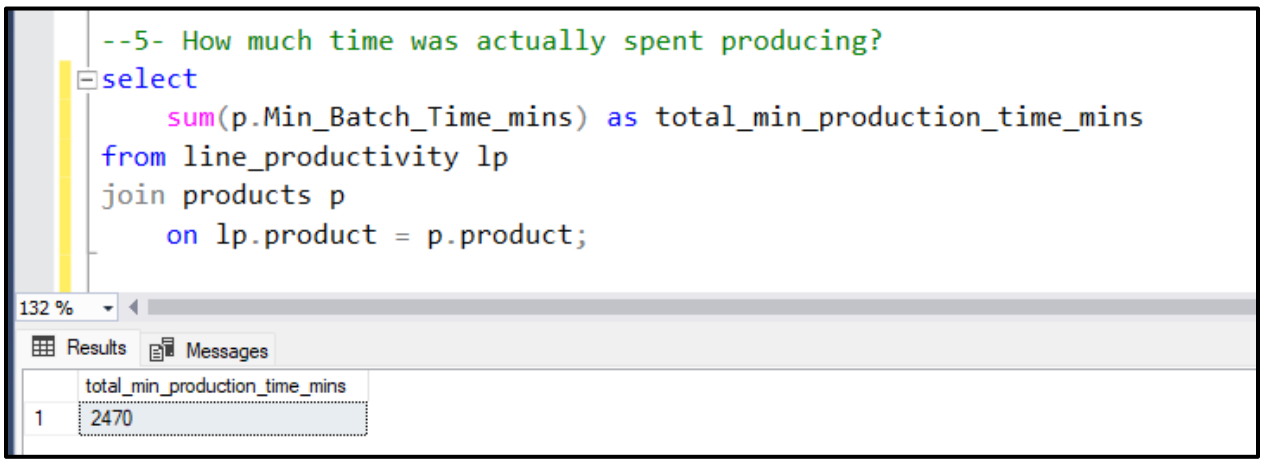


Figure 4-14 Analyze Q5

```
# How much time was actually spent producing?
total_Actualtime = Line_productivity['Actual time'].sum()
total_downtime=Line_downtime_long['Downtime_Minutes'].sum()
total_productive_time = total_Actualtime- total_downtime
total_productive_time
```

```
np.int64(2470)
```

Figure 4-15 Analyze Q5

what is the percentage of lost production?

Product Lost %
0.359771903

Figure 4-16 Analyze Q6

```
--6- How does daily production volume change over time?
select Date, count(Batch) as Total_Batches
from Line_Productivity
group by date
order by date
```

132 %

Results | Messages

	Date	Total_Batches
1	2024-08-29	7
2	2024-08-30	12
3	2024-08-31	7
4	2024-09-02	11
5	2024-09-03	1

Figure 4-17 Analyze Q6

```
#what is the percentage of lost production?
Downtime_impact_percent = (Line_downtime_long['Downtime_Minutes'].sum()/ Line_productivity['Actual time'].sum()) * 100
Downtime_impact_percent
```

```
np.float64(35.97719025401763)
```

Figure 4-18 Analyze Q6

How does daily production volume change over time?

Row Labels	Count of Batch
8/29/2024	7
8/30/2024	12
8/31/2024	7
9/2/2024	11
9/3/2024	1
Grand Total	**38**

Figure 4-19 Analyze Q7

```
--7- What is the average batch duration per product and operator?
select Product,Operator,avg(Total_Minutes) as Avg_Batch_Duration_Minutes
from Line_Productivity
group by Product , Operator
order by Product, Operator
```

132 %

Results | Messages

	Product	Operator	Avg_Batch_Duration_Minutes
1	CO-2L	Charlie	161
2	CO-2L	Dennis	152
3	CO-2L	Mac	130
4	CO-600	Charlie	90
5	CO-600	Dee	91
6	CO-600	Dennis	98
7	DC-600	Dee	80
8	DC-600	Mac	91
9	LE-600	Charlie	73
10	LE-600	Mac	103
11	OR-600	Mac	135
12	RB-600	Dee	100
13	RB-600	Dennis	91

Figure 4-20 Analyze Q7

```
#How does daily production volume change over time?
daily_production = ( Line_productivity.groupby('Date')['Batch'].count()
                    .reset_index()
                    .sort_values('Date'))
daily_production
```

	Date	Batch
0	2024-08-29	7
1	2024-08-30	12
2	2024-08-31	7
3	2024-09-02	12

Figure 4-21 Analyze Q7

What is the min batch time, downtime, and total duration for each batch?

Row Labels	Sum of Sche	Sum of Downtime (min	Total Actual Duration (mins)
422111	60	75	135
422112	60	40	100
422113	60	50	110
422114	60	40	100
422115	60	24	84
422116	60		60
422117	60	15	75
422118	60	60	120
422119	60	25	85
422120	60	52	112
422121	60	15	75
422122	60	25	85
422123	60	73	133
422124	60	40	100
422125	60	20	80
422126	60	44	104
422127	60	23	83
422128	60	52	112
422129	60	15	75
422130	60	20	80
422131	60	30	90
422132	60		60
422133	60	20	80
422134	60	50	110
422135	60	45	105
422136	60		60
422137	60	45	105
422138	60	20	80
422139	60	35	95
422140	60	63	123
422141	60	7	67
422142	60	30	90
422143	60	58	118
422144	98	54	152
422145	98	22	120
422146	98	62	160
422147	98	107	205
422148	98	32	130
Grand Total	**2470**	**1388**	**3858**

Figure 4-22 Analyze Q8

```sql
--8- Which operator consistently completes batches closest to the Min Batch Time?
select Min_Batch_Time_mins, lp.Operator,round(avg(abs(lp.Total_Minutes - p.Min_Batch_Time_mins)), 2) as Avg_Deviation_Minutes
from Line_Productivity lp join Products p
on lp.Product = p.Product
group by lp.Operator , Min_Batch_Time_mins
order by Avg_Deviation_Minutes
```

132 %

Results | Messages

	Min_Batch_Time_mins	Operator	Avg_Deviation_Minutes
1	60	Charlie	24
2	98	Mac	32
3	60	Dee	33
4	60	Dennis	35
5	60	Mac	42
6	98	Dennis	54
7	98	Charlie	63

Figure 4-23 Analyze Q8

```
#What is the min batch time, downtime, and total duration for each batch?
merged = pd.merge(Line_downtime_long, Line_productivity, on="Batch", how="inner")
merged2 = pd.merge(merged,Products, on="Product",how="inner")
batch_summary = merged2.groupby('Batch').agg(
    Min_Batch_Time=('Min batch time', 'min'),
    Total_Downtime=('Downtime_Minutes', 'sum'),
    Total_Duration=('Actual time', 'first')
).reset_index()
batch_summary
```

	Batch	Min_Batch_Time	Total_Downtime	Total_Duration
0	422111	60	75	135
1	422112	60	40	100
2	422113	60	50	110
3	422114	60	40	100
4	422115	60	24	84
5	422117	60	15	75

Figure 4-24 Analyze Q8

What is the average batch duration per product and operator?

Average of Total Minutes	Column Label						
Row Labels	Cola (2L)	Cola (600 ml)	Diet Cola	Lemon lime	Orange	Root Berry	Grand Total
Charlie	161.6666667	90.8		73			105.2727273
Dee		91.16666667	80			100.75	93.63636364
Dennis	152	98.25				91.66666667	102.5
Mac	130		91.66666667	103.3333333	135		106.25
Grand Total	**153.4**	**92.93333333**	**88.75**	**88.16666667**	**135**	**96.85714286**	**101.5263158**

Figure 4-25 Analyze Q9

```sql
--9- Which product-flavor-size combination shows the highest deviation from ideal performance?
select p.Product, p.Flavor,p.Size, round(avg(abs(lp.Total_Minutes - p.Min_Batch_Time_mins)), 2) as Avg_Deviation_Minutes,
    max(abs(lp.Total_Minutes - p.Min_Batch_Time_mins)) as Max_Deviation_Minutes,
    count(lp.Batch) as Total_Batches
from Line_Productivity lp
join Products p
    on lp.Product = p.Product
group by p.Product, p.Flavor, p.Size
order by Avg_Deviation_Minutes desc
```

132 %

Results | Messages

	Product	Flavor	Size	Avg_Deviation_Minutes	Max_Deviation_Minutes	Total_Batches
1	OR-600	Orange	600 ml	75	75	1
2	CO-2L	Cola (2 L)	2 L	55	107	5
3	RB-600	Root Berry	600 ml	36	63	7
4	CO-600	Cola (600 ml)	600 ml	32	73	15
5	DC-600	Diet Cola	600 ml	28	50	4
6	LE-600	Lemon lime	600 ml	28	50	6

Figure 4-26 Analyze Q9

```python
#What is the average batch duration per product and operator?
avg_batch_duration = (
    Line_productivity.groupby(['Product', 'Operator'])['Actual time'].mean()
    .reset_index()
    .rename(columns={'Actual time': 'Avg_Batch_Duration_Minute'})
    .sort_values('Avg_Batch_Duration_Minute', ascending=False))
avg_batch_duration
```

	Product	Operator	Avg_Batch_Duration_Minute
0	CO-2L	Charlie	161.666667
1	CO-2L	Dennis	152.000000
10	OR-600	Mac	135.000000
2	CO-2L	Mac	130.000000
9	LE-600	Mac	103.333333
11	RB-600	Dee	100.750000
5	CO-600	Dennis	98.250000
7	DC-600	Mac	91.666667
12	RB-600	Dennis	91.666667
4	CO-600	Dee	91.166667
3	CO-600	Charlie	90.800000
6	DC-600	Dee	80.000000
8	LE-600	Charlie	73.000000

Figure 4-27 Analyze Q9

Which operator has the highest downtime?	
Row Labels	**Sum of Downtime (mins)**
Charlie	384
Grand Total	**384**

Figure 4-28 Analyze Q10

```sql
--10- Which products have the highest number of batches produced?
select Product,count(Batch) as Total_Batches
from Line_Productivity
group by Product
order by Total_Batches desc
```

132 %

Results | Messages

	Product	Total_Batches
1	CO-600	15
2	RB-600	7
3	LE-600	6
4	CO-2L	5
5	DC-600	4
6	OR-600	1

Figure 4-29 Analyze Q10

```python
#Which operator has the highest downtime?
batch_downtime = (
    Line_downtime_long.groupby('Batch', as_index=False)
    .agg({'Downtime_Minutes': 'sum'})
)
merged = pd.merge(Line_productivity, batch_downtime, on="Batch", how="left")

downtime_by_operator = ( merged.groupby('Operator')['Downtime_Minutes'].sum()
             .sort_values(ascending=False))
downtime_by_operator
```

```
Operator
Charlie    384.0
Dee        370.0
Mac        332.0
Dennis     302.0
Name: Downtime_Minutes, dtype: float64
```

Figure 4-30 Analyze Q10

Which product–flavor–size combination shows the highest deviation from ideal performance?

Row Labels	Average of Downtime (mins)
RB-600	23.45454545
Grand Total	**23.45454545**

Figure 4-31 Analyze Q11

```sql
--11- Which product has the most downtime? /
select top 1 LP.Product,sum(LD.Downtime_in_mins) as Total_Downtime
FROM Line_Productivity LP join Line_Downtime LD on  LP.Batch = LD.Batch
group by LP.Product
order by Total_Downtime desc
```

132 %

Results | Messages

	Product	Total_Downtime
1	CO-600	494

Figure 4-32 Analyze Q11

```python
#Which product-flavor-size combination shows the highest deviation from ideal performance?

Products["Deviation"] = abs(Line_productivity["Actual time"] - Products["Min batch time"])


product_flavor= (
    Products.groupby(["Product", "Flavor", "Size(ml)"], as_index=False)["Deviation"]
    .mean()
    .sort_values(by="Deviation", ascending=False)
    .head(1)
)

product_flavor
```

	Product	Flavor	Size(ml)	Deviation
4	OR-600	Orange	600	75.0

Figure 4-33 Analyze Q11

Which products have the highest number of batches produced?

Row Labels	Count of Batch
CO-600	15
Grand Total	**15**

Figure 4-34 Analyze Q12

```sql
--12- Which product consumes the most production time?
select top 1  Product, sum(Total_Minutes) as Total_Production_Time
from Line_Productivity
group by Product
order by Total_Production_Time desc
```

132 %

Results | Messages

	Product	Total_Production_Time
1	CO-600	1394

Figure 4-35 Analyze Q12

```python
#Which products have the highest number of batches produced?
batches_per_product = (Line_productivity.groupby('Product')['Batch'].nunique()
                       .reset_index(name='Number_of_Batches')
                       .sort_values('Number_of_Batches', ascending=False))
batches_per_product
```

	Product	Number_of_Batches
1	CO-600	15
5	RB-600	7
3	LE-600	6
0	CO-2L	5
2	DC-600	4
4	OR-600	1

Figure 4-36 Analyze Q12

Which products have the most	
Row Labels	**Total Downtime**
CO-600	494
Grand Total	**494**

Figure 4-37 Analyze Q13

```sql
--13- What is the percentage of downtime impact per product?
select LP.Product,sum (LD.Downtime_in_mins) as Total_Downtime, sum(LP.Total_Minutes) as Total_Production_Time,
    cast(sum(LD.Downtime_in_mins) * 100.0 / sum(LP.Total_Minutes) as decimal (5,2)) as Downtime_Impact_Percentage
from Line_Productivity LP join Line_Downtime LD on LP.Batch = LD.Batch
group by LP.Product
order by Downtime_Impact_Percentage desc
```

132 %

Results | Messages

	Product	Total_Downtime	Total_Production_Time	Downtime_Impact_Percentage
1	OR-600	75	270	27.78
2	RB-600	258	1119	23.06
3	LE-600	169	744	22.72
4	DC-600	115	510	22.55
5	CO-600	494	2433	20.30
6	CO-2L	277	1779	15.57

Figure 4-38 Analyze Q13

```python
# Which products have the most downtime?
downtime_by_product = (merged.groupby('Product')['Downtime_Minutes'].sum()
                       .sort_values(ascending=False))
downtime_by_product
```

```
Product
CO-600    494.0
CO-2L     277.0
RB-600    258.0
LE-600    169.0
DC-600    115.0
OR-600     75.0
Name: Downtime_Minutes, dtype: float64
```

Figure 4-39 Analyze Q13

Which products has the most production time?

Row Labels	Total Actual Duration (mins)
CO-600	1394
Grand Total	**1394**

Figure 4-40 Analyze Q14

```
--14- Which operator has the highest downtime?
select top 1 LP.Operator,sum(LD.Downtime_in_mins) as Total_Downtime
from Line_Productivity LP join Line_Downtime LD on LP.Batch = LD.Batch
group by LP.Operator
order by Total_Downtime desc
```

132 %

Results | Messages

	Operator	Total_Downtime
1	Charlie	384

Figure 4-41 Analyze Q14

```
#Which products consume the most production time?
production_time_by_product = (Line_productivity.groupby('Product')['Actual time'].sum()
                              .reset_index()
                              .sort_values('Actual time', ascending=False))
production_time_by_product
```

	Product	Actual time
1	CO-600	1394
0	CO-2L	767
5	RB-600	678
3	LE-600	529
2	DC-600	355
4	OR-600	135

Figure 4-42 Analyze Q14

What is the downtime % per product?

Row Labels	Downtime Impact by Product
CO-2L	0.07179886
CO-600	0.128045619
DC-600	0.029808191
LE-600	0.04380508
OR-600	0.019440124
RB-600	0.066874028
Grand Total	**0.359771903**

Figure 4-43 Analyze Q15

```sql
--15- How many batches does each operator handle?
select operator,count(batch) as total_batches
from line_productivity
group by operator
order by total_batches desc
```

132 %

Results | Messages

	operator	total_batches
1	Charlie	11
2	Dee	11
3	Dennis	8
4	Mac	8

Figure 4-44 Analyze Q15

```python
#What is the downtime % per product?
batch_downtime = (
    Line_downtime_long.groupby('Batch', as_index=False)
    .agg({'Downtime_Minutes': 'sum'})
)

merged = pd.merge(Line_productivity, batch_downtime, on='Batch', how='left')

downtime_impact = (
    merged.groupby('Product', as_index=False)
    .agg({
        'Downtime_Minutes': 'sum',
        'Actual time': 'sum'
    })
)

downtime_impact['Downtime Impact %'] = (
    downtime_impact['Downtime_Minutes'] / downtime_impact['Actual time'] * 100
)


downtime_impact = downtime_impact.sort_values('Downtime Impact %', ascending=False)
```

```python
downtime_impact
```

	Product	Downtime_Minutes	Actual time	Downtime Impact %
4	OR-600	75.0	135	55.555556
5	RB-600	258.0	678	38.053097
0	CO-2L	277.0	767	36.114733
1	CO-600	494.0	1394	35.437590
2	DC-600	115.0	355	32.394366
3	LE-600	169.0	529	31.947070

Figure 4-45 Analyze Q15

How many batches does each operator handle?

Row Labels	Count of Batch
Charlie	11
Dee	11
Dennis	8
Mac	8
Grand Total	**38**

Figure 4-46 Analyze Q17

```sql
--17- Is most of the downtime caused by operators or by non-operator?
select ld.operator_error,sum(ld.downtime_in_mins) as total_downtime,
cast(sum(ld.downtime_in_mins) * 100.0 / (select sum(downtime_in_mins) from line_downtime) as decimal(5,2)) as downtime_percentage
from line_downtime ld
group by ld.operator_error
```

132 %

Results | Messages

	operator_error	total_downtime	downtime_percentage
1	No	612	44.09
2	Yes	776	55.91

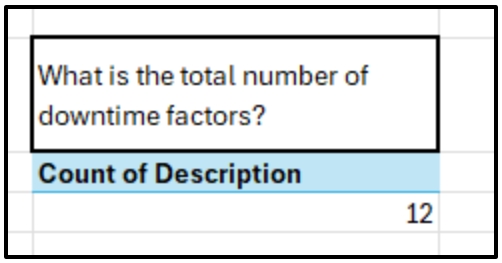
Figure 4-47 Analyze Q17

```python
#How many batches does each operator handle?
batches_per_operator = (
    Line_productivity.groupby('Operator')['Batch'].nunique()
                     .reset_index(name='Number_of_Batches')
                     .sort_values('Number_of_Batches', ascending=False))

batches_per_operator
```

	Operator	Number_of_Batches
0	Charlie	11
1	Dee	11
2	Dennis	8
3	Mac	8

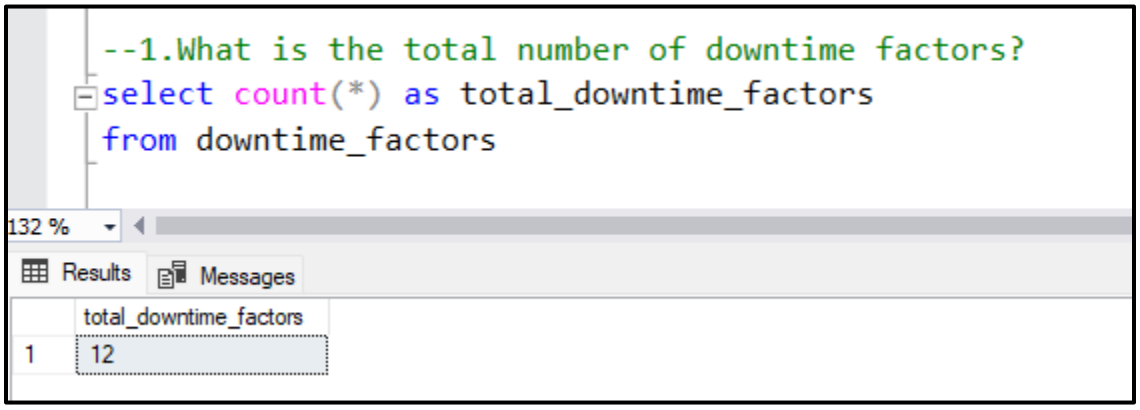
Figure 4-48 Analyze Q17



What is the total number of downtime factors?

Count of Description
12

Figure 4-49 Analyze Q20



```
--1.What is the total number of downtime factors?
select count(*) as total_downtime_factors
from downtime_factors
```

132 %

Results | Messages

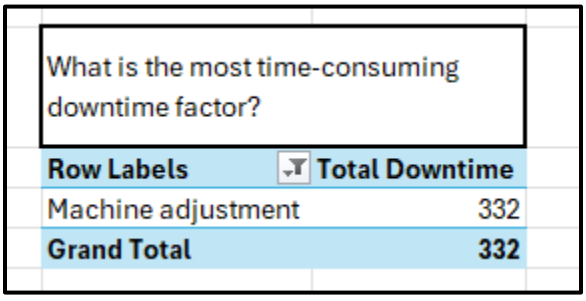
	total_downtime_factors
1	12

Figure 4-50 Analyze Q20

```
#What is the total number of downtime factors?
Downtime_factors['Description'].count()
```

```
np.int64(12)
```

Figure 4-51 Analyze Q20



What is the most time-consuming downtime factor?

Row Labels	Total Downtime
Machine adjustment	332
Grand Total	**332**

Figure 4-52 Analyze Q21

```sql
--2. What is the most time consuming by downtime factor?
select top 1 ld.factor,sum(ld.downtime_in_mins) as total_downtime
from line_downtime ld
group by ld.factor
order by total_downtime desc
```

132 %

Results | Messages

	factor	total_downtime
1	6	332

Figure 4-53 Analyze Q21

```python
# What is the most time consuming by downtime factor?
factor_downtime = (
 merged2.groupby('Description', as_index=False)
    .agg({'Downtime_Minutes': 'sum'})
)

factor_downtime = factor_downtime.sort_values('Downtime_Minutes', ascending=False).head(1)
factor_downtime
```

	Description	Downtime_Minutes
7	Machine adjustment	332

Figure 4-54 Analyze Q21

What is the most occurred downtime factor?

Row Labels	Count of Downtime
Machine adjustment	12
Grand Total	**12**

Figure 4-55 Analyze Q22

```
--3.What is the most occurred downtime factor?
select top 1 ld.factor,Description,count(*) as occurrences
from line_downtime ld
group by ld.factor ,Description
order by occurrences desc
```

132 %

Results | Messages

	factor	Description	occurrences
1	6	Machine adjustment	12

Figure 4-56 Analyze Q22

```
# What is the most occurred downtime factor?

factor_counts = (
  merged2.groupby('Description', as_index=False)
    .size()
    .rename(columns={'size': 'Count'})
)
most_occurred_factor = factor_counts.sort_values('Count', ascending=False).head(1)
most_occurred_factor
```

	Description	Count
7	Machine adjustment	12

Figure 4-57 Analyze Q22

Are certain products more affected by specific downtime factors?

Sum of Downtime (m Column Labels							
Row Labels	Cola (2L)	Cola (600 ml)	Diet Cola	Lemon lime	Orange	Root Berry	Grand Total
Batch change		20		80	60		160
Batch coding error	31	44	20	20		30	145
Calibration error		10		24		15	49
Conveyor belt jam		17					17
Inventory shortage	42	108	30	25		20	225
Label switch		20				13	33
Labeling error	22					20	42
Machine adjustment	120	57		20		135	332
Machine failure	55	116	50		15	18	254
Other	7	45	15			7	74
Product spill		57					57
Grand Total	**277**	**494**	**115**	**169**	**75**	**258**	**1388**

Figure 4-58 Analyze Q24

```
--5. Are certain products more affected by specific downtime factors?
select lp.product,ld.factor,sum(ld.downtime_in_mins) as total_downtime
from line_productivity lp join line_downtime ld on lp.batch = ld.batch
group by lp.product, ld.factor
order by lp.product, total_downtime desc
```

132 %

Results | Messages

	product	factor	total_downtime
1	CO-2L	6	120
2	CO-2L	7	55
3	CO-2L	4	42
4	CO-2L	8	31
5	CO-2L	3	22
6	CO-2L	12	7
7	CO-600	7	116
8	CO-600	4	108
9	CO-600	6	57
10	CO-600	5	57
11	CO-600	12	45
12	CO-600	8	44
13	CO-600	11	20
14	CO-600	2	20
15	CO-600	9	17

Figure 4-59 Analyze Q24

```python
#Are certain products more affected by specific downtime factors?
merged_final = (
    Line_downtime_long
    .merge(Line_productivity[['Batch', 'Product', 'Date', 'Shift']], on='Batch', how='inner')
    .merge(Downtime_factors[['Factor', 'Description']], on='Factor', how='left')
)

product_factor_impact = (
    merged_final.groupby(['Product', 'Description'], as_index=False)['Downtime_Minutes']
    .sum()
    .sort_values(['Product', 'Downtime_Minutes'], ascending=[True, False])
)

product_factor_impact['Total_Product_Downtime'] = (
    product_factor_impact.groupby('Product')['Downtime_Minutes'].transform('sum')
)
product_factor_impact['Downtime_%'] = (
    product_factor_impact['Downtime_Minutes'] / product_factor_impact['Total_Product_Downtime'] * 100
)

product_factor_impact
```

	Product	Description	Downtime_Minutes	Total_Product_Downtime	Downtime_%
3	CO-2L	Machine adjustment	120	277	43.321300
4	CO-2L	Machine failure	55	277	19.855596
1	CO-2L	Inventory shortage	42	277	15.162455
0	CO-2L	Batch coding error	31	277	11.191336

Figure 4-60 Analyze Q24

Which downtime factors affect each product the most?

Sum of Downtime (m Column Labels							
Row Labels	Cola (2L)	Cola (600 ml)	Diet Cola	Lemon lime	Orange	Root Berry	Grand Total
Machine adjustment	120	57		20		135	332
Machine failure	55	116	50		15	18	254
Inventory shortage	42	108	30	25		20	225
Batch change		20		80	60		160
Batch coding error	31	44	20	20		30	145
Other	7	45	15			7	74
Product spill		57					57
Calibration error		10		24		15	49
Labeling error	22					20	42
Label switch		20				13	33
Conveyor belt jam		17					17
Grand Total	**277**	**494**	**115**	**169**	**75**	**258**	**1388**

Figure 4-61 Analyze Q25

```
--6. Which operators face more operator-related downtime factors?
select lp.operator,count(ld.factor) as operator_related_downtime_events,sum(ld.downtime_in_mins) as total_operator_related_downtime
from line_productivity lp join line_downtime ld on lp.batch = ld.batch
where ld.operator_error = 'Yes'
group by lp.operator
order by total_operator_related_downtime desc
```

120 %

Results | Messages

	operator	operator_related_downtime_events	total_operator_related_downtime
1	Charlie	9	228
2	Dee	10	192
3	Mac	7	192
4	Dennis	6	164

Figure 4-62 Analyze Q25

```python
# Which downtime factors affect each product the most?
product_factor = (
    merged_final.groupby(['Product', 'Description'], as_index=False)['Downtime_Minutes']
    .sum()
)

product_factor['Rank'] = (
    product_factor.groupby('Product')['Downtime_Minutes'].rank(method='first', ascending=False)
)

top_factor_per_product = product_factor[product_factor['Rank'] == 1].drop(columns='Rank')

top_factor_per_product = top_factor_per_product.sort_values('Downtime_Minutes', ascending=False)

top_factor_per_product
```

	Product	Description	Downtime_Minutes
32	RB-600	Machine adjustment	135
3	CO-2L	Machine adjustment	120
13	CO-600	Machine failure	116
20	LE-600	Batch change	80
25	OR-600	Batch change	60
18	DC-600	Machine failure	50

Figure 4-63 Analyze Q25

Which downtime factors occur most often in each shift?

Count of Description	Column Labels			
Row Labels	Afternoon	Morning	Night	Grand Total
Batch change	3	2		5
Batch coding error	2	2	2	6
Calibration error	1	1	1	3
Conveyor belt jam		1		1
Inventory shortage	3	4	2	9
Label switch	1		2	3
Labeling error	1		1	2
Machine adjustment	6	3	3	12
Machine failure	3	7	1	11
Other	3	1	2	6
Product spill	2	1		3
Grand Total	**25**	**22**	**14**	**61**

Figure 4-64 Analyze Q27

```sql
--7. Which downtime factors occur most often in each shift?
select  lp.shifts,ld.factor, ld.description,
 count(distinct lp.batch) as most_occurrences
from line_productivity lp join line_downtime ld on lp.batch = ld.batch
group by lp.shifts, ld.factor, ld.description
order by lp.shifts, most_occurrences desc
```

120 %

Results | Messages

	shifts	factor	description	most_occurrences
1	Afternoon	6	Machine adjustment	6
2	Afternoon	7	Machine failure	3
3	Afternoon	12	Other	3
4	Afternoon	2	Batch change	3
5	Afternoon	4	Inventory shortage	3
6	Afternoon	5	Product spill	2
7	Afternoon	8	Batch coding error	2
8	Afternoon	3	Labeling error	1
9	Afternoon	10	Calibration error	1
10	Afternoon	11	Label switch	1
11	Evening	6	Machine adjustment	3
12	Evening	8	Batch coding error	2

Figure 4-65 Analyze Q27

```
#Which downtime factors occur most often in each shift?
merged4 = (
    Line_downtime_long
    .merge(Line_productivity[['Batch', 'Operator', 'Date', 'Shift']],
           on='Batch', how='inner')
    .merge(Downtime_factors[['Factor', 'Description', 'Operator Error']],
           on='Factor', how='left')
)

downtime_by_shift_factor = (merged4.groupby(['Date','Shift', 'Description']).size()
                            .reset_index(name='Count')
                            .sort_values(['Shift', 'Count'], ascending=[True, False]))
downtime_by_shift_factor
```

	Date	Shift	Description	Count
0	2024-08-29	Afternoon	Batch change	3
4	2024-08-29	Afternoon	Machine adjustment	2
9	2024-08-30	Afternoon	Machine adjustment	2
12	2024-08-30	Afternoon	Product spill	2
30	2024-09-02	Afternoon	Machine adjustment	2
31	2024-09-02	Afternoon	Machine failure	2

Figure 4-66 Analyze Q27

4.3 Summary

In this phase, the analysis was conducted using three main tools: Excel, SQL, and Python. Each software was used to explore and analyze downtime data by product, operator, shift, and factor, ensuring consistency and validation across different platforms. The results from these analyses provided a comprehensive understanding of the dataset and revealed key patterns related to production efficiency and downtime causes. The insights gained here form the foundation for the next phase, where visualization dashboards will be developed using Tableau and Power BI to present the findings in an interactive and accessible way.

5 Phase THREE: Forecasting Questions

5.1 Regression model

A regression model analyzes how a dependent variable changes in relation to independent variables. In this study, it was used to examine how operator, product type, batch characteristics, and shift influence total downtime in the production line. The following subheadings present several models developed in Excel to test different predictor combinations and identify which factors most affect downtime.

5.1.1 Model 1

Predictors used:

All variables: Operators (Mac, Charlie, Dee) + Products (OR-600, LE-600, CO-600, DC-600, RB-600) + Size + MinBatchTime + Shifts (Morning, Afternoon)

Results:

- $R^2 = 0.25$ Adjusted $R^2 = -0.10$
- Significance F = 0.57, not significant
- Only one variable (LE-600) showed a noticeable negative coefficient (–14.37) but was not statistically significant ($p > 0.05$).
- Several columns caused errors (#NUM!) because they had no variation (e.g., MinBatchTime, RB-600).

Interpretation:

Including all variables made the model unstable and statistically weak. The model explained only ~25% of the variation in downtime, with no strong predictors. Too many columns for a small dataset (38 samples) reduced reliability.

Regression Statistics	
Multiple R	0.498387179
R Square	0.24838978
Adjusted R Square	-0.10405845
Standard Error	23.47929996
Observations	38

ANOVA

	df	*SS*	*MS*	*F*	*Significance F*
Regression	12	4918.980471	409.9150393	0.892287502	0.565931671
Residual	27	14884.49321	551.2775264		
Total	39	19803.47368			

	Coefficients	*Standard Error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*	*Lower 95.0%*	*Upper 95.0%*
Intercept	30.3988356	17.40196666	1.746862076	0.092029391	-5.30705063	66.10472183	-5.30705063	66.10472183
Mac	0.523984013	21.34595671	0.024547226	0.980596641	-43.27430136	44.32226939	-43.27430136	44.32226939
Charlie	-13.08364639	18.23306944	-0.717577829	0.479180575	-50.49481467	24.3275219	-50.49481467	24.3275219
Dee	-0.309522123	13.11479243	-0.023600993	0.981344444	-27.21885346	26.59980921	-27.21885346	26.59980921
OR-600	39.5360662	34.4530627	1.147534155	0.261227718	-31.15577923	110.2279116	-31.15577923	110.2279116
LE-600	-14.3698344	20.35264245	-0.706042689	0.486209697	-56.13000727	27.39033847	-56.13000727	27.39033847
CO-600	-2.63566763	13.13632177	-0.200639698	0.842484038	-29.58917352	24.31783826	-29.58917352	24.31783826
DC-600	-13.4437485	22.53144848	-0.596665967	0.555700846	-59.67446207	32.78696507	-59.67446207	32.78696507
RB-600	0	0	65535	#NUM!	0	0	0	0
Size	0.013676207	0.012360968	1.106402624	#NUM!	-0.011686404	0.039038817	-0.011686404	0.039038817
MinBatchTime	0	0	65535	#NUM!	0	0	0	0
Morning	-3.664609998	15.0994317	-0.242698538	#NUM!	-34.64608474	27.31686474	-34.64608474	27.31686474
Afternoon	10.21177247	16.6173929	0.61452314	0.544013992	-23.88430139	44.30784633	-23.88430139	44.30784633

Figure 5-1 Regression model 1

5.1.2 Model 2

Predictors used:

Operators (Mac, Charlie, Dee) + Products (LE-600, CO-600, DC-600, RB-600) + Shifts (Morning, Afternoon)

Removed: Cleaned Model (Removed Size, MinBatchTime, Orange Batch).

Results:

- $R^2 = 0.19$ Adjusted $R^2 = -0.09$
- Significance F = 0.72, not significant
- **LE-600 became statistically significant (p = 0.046)** with a coefficient of –33.5, meaning around 33 minutes less downtime than the baseline product.
- Other predictors remained insignificant.

Interpretation:

Cleaning the dataset improved stability and revealed one meaningful relationship — the product LE-600 consistently had significantly lower downtime compared to the baseline product (OR-600). Other products and shifts still showed no strong effect.

SUMMARY OUTPUT

Regression Statistics	
Multiple R	0.43115457
R Square	0.185894263
Adjusted R Square	-0.085474316
Standard Error	23.47929996
Observations	37

ANOVA

	df	*SS*	*MS*	*F*	*Significance F*
Regression	9	3398.75003	377.6388923	0.685025009	0.715807714
Residual	27	14884.49321	551.2775264		
Total	36	18283.24324			

	Coefficients	*Standard Error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*	*Lower 95.0%*	*Upper 95.0%*
Intercept	57.75124955	21.84562237	2.643607428	0.013492767	12.92773493	102.5747642	12.92773493	102.5747642
Mac	0.523984013	21.34595671	0.024547226	0.980596641	-43.27430136	44.32226939	-43.27430136	44.32226939
Charlie	-13.08364639	18.23306944	-0.717577829	0.479180575	-50.49481467	24.3275219	-50.49481467	24.3275219
Dee	-0.309522123	13.11479243	-0.023600993	0.981344444	-27.21885346	26.59980921	-27.21885346	26.59980921
LE-600	-33.51652416	15.99044912	-2.096033946	0.045587199	-66.32621564	-0.706832684	-66.32621564	-0.706832684
CO-600	-21.78235739	13.29201772	-1.638754767	0.112866724	-49.05532497	5.490610187	-49.05532497	5.490610187
DC-600	-32.59043826	18.95260254	-1.719575884	0.09695503	-71.47796652	6.297089997	-71.47796652	6.297089997
RB-600	-19.14668976	17.30535462	-1.106402624	0.278313077	-54.65434448	16.36096495	-54.65434448	16.36096495
Morning	-3.664609998	15.0994317	-0.242698538	0.810073177	-34.64608474	27.31686474	-34.64608474	27.31686474
Afternoon	10.21177247	16.6173929	0.61452314	0.544013992	-23.88430139	44.30784633	-23.88430139	44.30784633

Figure 5-2 Regression model 2

5.1.3 Model 3

Predictors used:

Operators (Mac, Charlie, Dee) + Products (LE-600, CO-600, DC-600, RB-600)

Results:

- $R^2 = 0.16$ Adjusted $R^2 = -0.04$
- Significance F = 0.60, not significant
- **LE-600 remained significant (p = 0.049) with coefficient –29.98.**
- DC-600 and CO-600 were borderline significant (p ≈ 0.07–0.11).
- Operator variables showed no effect (p > 0.6).

Interpretation:

Product type was the main factor influencing downtime. **LE-600 showed significantly lower downtime (~30 minutes less)**, while other products might also reduce downtime but with weaker evidence. Operators had no measurable impact.

SUMMARY OUTPUT

Regression Statistics	
Multiple R	0.400438614
R Square	0.160351083
Adjusted R Square	-0.042322793
Standard Error	23.00787375
Observations	37

ANOVA

	df	*SS*	*MS*	*F*	*Significance F*
Regression	7	2931.737862	418.8196945	0.791177858	0.600569366
Residual	29	15351.50538	529.3622545		
Total	36	18283.24324			

	Coefficients	*Standard Error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*	*Lower 95.0%*	*Upper 95.0%*
Intercept	55.1815562	14.13667898	3.903431369	0.000519681	26.2688013	84.09431109	26.2688013	84.09431109
Mac	8.352901132	16.55338476	0.504603817	0.617650053	-25.50257205	42.20837432	-25.50257205	42.20837432
Charlie	-2.420227371	12.27982849	-0.197089672	0.845132213	-27.5352966	22.69484186	-27.5352966	22.69484186
Dee	-0.388232091	10.84660021	-0.035792975	0.971692679	-22.57202035	21.79555617	-22.57202035	21.79555617
LE-600	-29.98122641	14.59750172	-2.053860105	0.049102553	-59.83646962	-0.125983198	-59.83646962	-0.125983198
CO-600	-21.28618757	12.99740335	-1.637726167	0.112285741	-47.86886216	5.296487025	-47.86886216	5.296487025
DC-600	-32.59917402	17.45157219	-1.867979209	0.071900003	-68.29164678	3.093298731	-68.29164678	3.093298731
RB-600	-18.10256643	15.90167287	-1.138406416	0.26426834	-50.62513915	14.42000629	-50.62513915	14.42000629

Figure 5-3 Regression model 3

5.1.4 Model 4

Predictors used:

Operators (Mac, Charlie, Dee) + Shifts (Morning, Afternoon)

Results:

- $R^2 = 0.03$ Adjusted $R^2 = -0.13$
- Significance F = 0.97, not significant
- All p-values > 0.4 → no variable significant.

Interpretation:

Neither the **operator** nor the **shift** significantly affected downtime. The model explained almost none of the variation (3%), showing that **human and time-of-day factors are not the main reason for downtime**.

SUMMARY OUTPUT

Regression Statistics	
Multiple R	0.169333613
R Square	0.028673872
Adjusted R Squa	-0.127991632
Standard Error	23.93471757
Observations	37

ANOVA

	df	*SS*	*MS*	*F*	*Significance F*
Regression	5	524.2513846	104.8502769	0.183026075	0.966906762
Residual	31	17758.99186	572.8707051		
Total	36	18283.24324			

	Coefficients	*Standard Error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*	*Lower 95.0%*	*Upper 95.0%*
Intercept	46.67351992	14.53529564	3.21104717	0.003075219	17.02858903	76.31845082	17.02858903	76.31845082
Mac	-7.973737525	15.67319583	-0.508749946	0.614529467	-39.93943117	23.99195612	-39.93943117	23.99195612
Charlie	-11.07188414	16.44149845	-0.673410892	0.505675926	-44.60454132	22.46077303	-44.60454132	22.46077303
Dee	-7.533803557	12.26230208	-0.614387372	0.543442641	-32.54293354	17.47532643	-32.54293354	17.47532643
Morning	-10.08948	12.48836901	-0.807910144	0.42529873	-35.55967652	15.38071651	-35.55967652	15.38071651
Afternoon	-0.761799355	14.71336997	-0.051775994	0.959039384	-30.76991524	29.24631653	-30.76991524	29.24631653

Figure 5-4 Regression model 4

5.1.5 Model 5

Predictors used:

Products (LE-600, CO-600, DC-600, RB-600) + Shifts (Morning, Afternoon)

Results:

- $R^2 = 0.15$ Adjusted $R^2 = -0.02$
- Significance F = 0.52, not significant
- **LE-600 nearly significant (p = 0.053)** → ~29 minutes less downtime than baseline.
- CO-600 (p = 0.094) and DC-600 (p = 0.107) possibly reduce downtime too.
- Morning and Afternoon shifts had no effect (p > 0.7).

Interpretation:

Downtime is mainly linked to **product type**, not shift timing. **LE-600 batches have consistently lower downtime**, and CO-600 / DC-600 might show similar patterns if more data were added. Shift timing shows no statistical influence.

SUMMARY OUTPUT

Regression Statistics	
Multiple R	0.387031205
R Square	0.149793153
Adjusted R Square	-0.020248216
Standard Error	22.76293668
Observations	37

ANOVA

	df	*SS*	*MS*	*F*	*Significance F*
Regression	6	2738.704661	456.4507768	0.880921825	0.520733641
Residual	30	15544.53858	518.1512861		
Total	36	18283.24324			

	Coefficients	*Standard Error*	*t Stat*	*P-value*	*Lower 95%*	*Upper 95%*	*Lower 95.0%*	*Upper 95.0%*
Intercept	53.75132643	13.95817511	3.850884949	0.00057429	25.24492987	82.25772299	25.24492987	82.25772299
LE-600	-28.95584355	14.37048631	-2.014952238	0.052947919	-58.30429193	0.392604827	-58.30429193	0.392604827
CO-600	-20.94428981	12.12303681	-1.727643835	0.09433526	-45.70283398	3.814254355	-45.70283398	3.814254355
DC-600	-25.75182656	15.51157213	-1.660168701	0.107299142	-57.43068308	5.927029956	-57.43068308	5.927029956
RB-600	-16.09267635	14.78298433	-1.088594562	0.285000858	-46.28355807	14.09820537	-46.28355807	14.09820537
Morning	-1.870183526	11.44218836	-0.163446315	0.871263168	-25.23824966	21.49788261	-25.23824966	21.49788261
Afternoon	3.371183787	13.09523677	0.257435879	0.798599943	-23.37285757	30.11522515	-23.37285757	30.11522515

Figure 5-5 Regression model 5

5.2 Summary

Across the five regression models, different combinations of predictors were tested to identify factors influencing production downtime. The results showed that **product type** had the most consistent effect, with **LE-600** repeatedly associated with lower downtime by around 30 minutes compared to the baseline product. Other products (CO-600 and DC-600) showed possible but weaker effects, while **operators and shift timing** had no significant influence. All models had relatively low **R^2 values (15–25%)**, indicating that downtime is also affected by other factors not included in the current dataset, such as machine conditions or operational interruptions.

6 Phase FOUR: Visualization

6.1 Introduction

The SMART analysis questions developed across five areas — **Overview, Production Line Performance, Operator Performance, Product Analysis, and Downtime Factors** — were translated into interactive Tableau dashboards. The goal of these dashboards is to visually summarize key production metrics, highlight downtime patterns, and reveal the major causes of production losses. By combining KPIs, trend visualizations, and cause-based breakdowns, the dashboards allow decision-makers to quickly assess performance, identify inefficiencies, and take data-driven corrective action.

To present insights clearly, the visualization phase was structured into two main dashboards: **Downtime Analysis** and **Downtime Cause Analysis.**

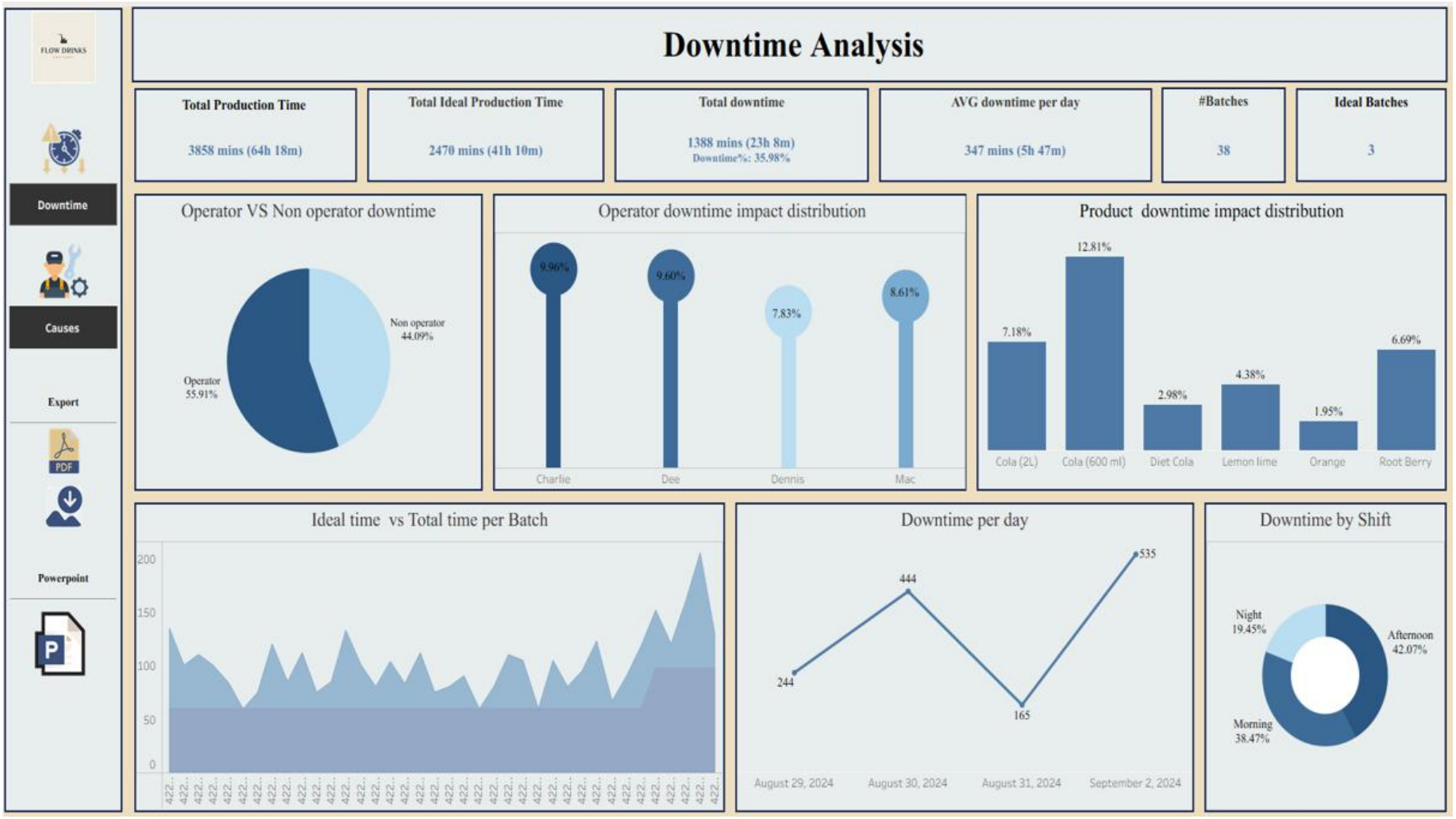


Figure 6-1 Downtime Analysis Dashboard

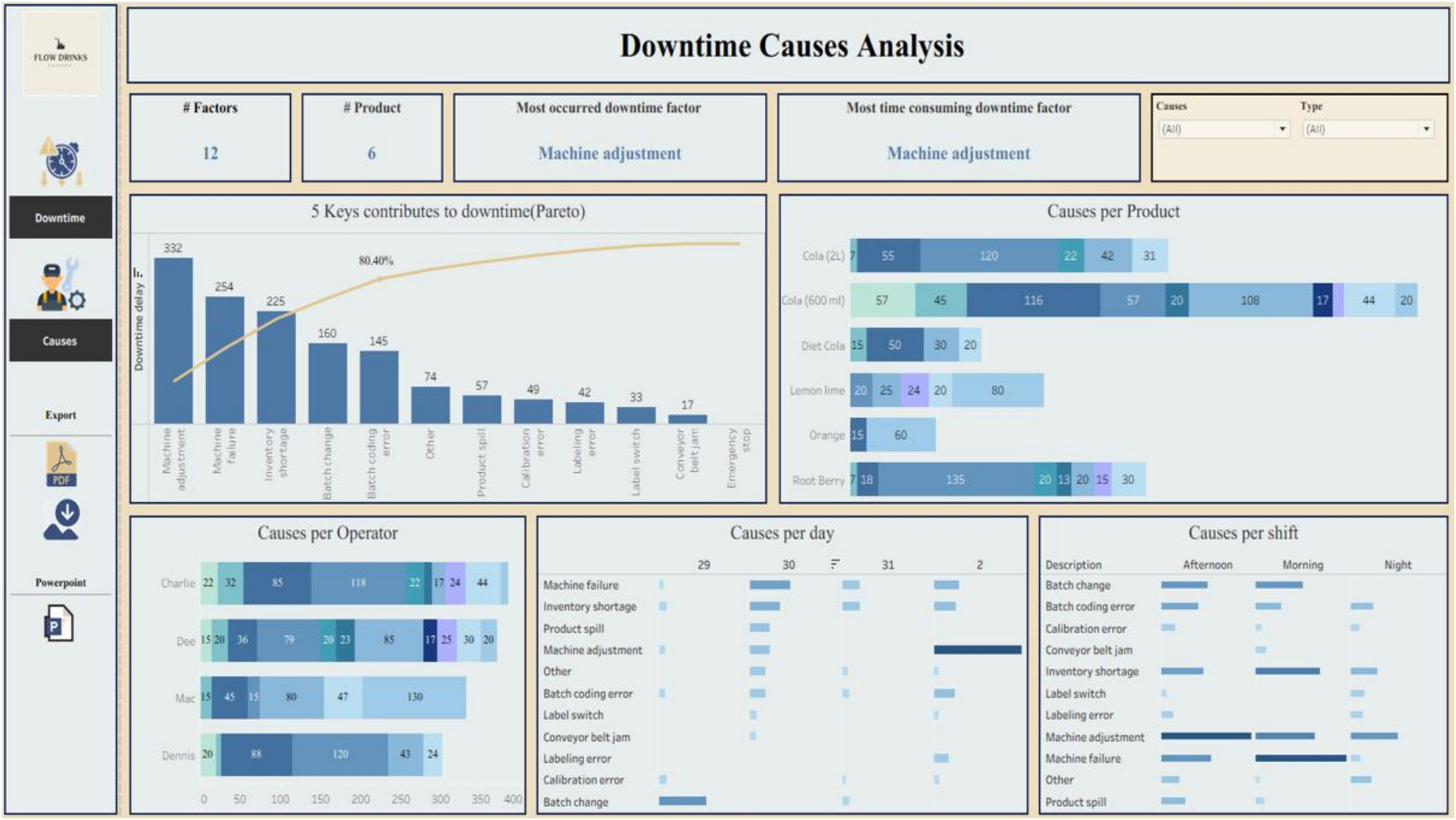


Figure 6-2 Downtime Cause Analysis Dashboard

6.2 Dashboard Design & Structure

The visualization design followed a clear and intuitive layout to ensure readability and quick interpretation. Two dashboards were created:

- **Downtime Analysis Dashboard**: provides an overall performance summary, showing total production time, downtime duration, downtime impact, and batch-level comparisons.
- **Downtime Cause Analysis Dashboard**: focuses on identifying and ranking the main contributors to downtime using a Pareto chart, along with detailed breakdowns by product, operator, shift, and day.

Both dashboards include interactive filters (e.g., by date or factor type) to allow users to drill down into specific time periods or categories. Consistent color schemes and chart formatting were applied to ensure visual coherence across all components.

6.3 Dashboards Components

6.3.1 Downtime Analysis Dashboard

The dashboard is divided into three rows:

- **Executive Summary Metrics (Top Row):**

Total Production Time: Total actual time the line ran
Total Ideal Production Time: Total time the line was scheduled to run.
Total downtime: The total time lost, with the **Downtime %** calculated as (1388 / 3858).
AVG downtime per day (): Average time lost each day.
#Batches: Total number of batches.
Ideal Batches: Total number of batches that weren't subjected to downtime.

- **Impact Distribution (Middle Row):**

Operator VS Non operator downtime: A pie chart showing the distribution of downtime among operator-related factors, and non-operator-related factors
Operator downtime impact distribution: A bar chart showing the relative impact of the total operator-attributed downtime by each individual operators.
Product downtime impact distribution: A bar chart showing the percentage of total downtime attributed to each product.

- **Trend and Periodicity (Bottom Row):**

Ideal time vs Total time per Batch: An area chart showing the trend of ideal production time versus total production time over many batches, illustrating variability.
Downtime per day: A line chart showing the trend of daily downtime.
Downtime by shift: A donut chart showing the distribution of total downtime among the three daily shifts, Morning, Afternoon, and Night.

6.3.2 Downtime Cause Analysis Dashboard

The dashboard is organized into four main rows of information:

- **Key Metrics (Top Row):**

Factors: The total number of downtime factors.
Product: The total number of prodcuts
Most occurred downtime factor: The downtime factor that happened most frequently.
Most time-consuming downtime factor: The downtime factor that resulted in the longest duration of downtime.
Filters: Dropdowns to filter the view by **Causes** and **Type of downtime factors**.

- **Pareto Analysis & Product Causes (Second Row):**

5 Keys contribute to downtime (Pareto): A bar chart (Pareto chart) showing the duration of the top 5 downtime factors and their cumulative contribution.
Causes per Product: A stacked bar chart showing the breakdown of downtime causes for each product.

- **Operator and Daily Causes (Third Row):**

Causes per Operator: A stacked bar chart showing which causes contributes to the downtime experienced by each operator.

- **Shift Causes (Bottom Right):**

Causes per day: A heat map/matrix chart showing which downtime causes are most prevalent during each day.
Causes per shift: A heat map/matrix chart showing which downtime causes are most prevalent during shifts.

6.4 Summary

This chapter presented the visualization phase of the project, where Tableau was used to transform analytical results into clear and interactive dashboards. Two main dashboards were developed — **Downtime Analysis** and **Downtime Causes** — each serving a distinct purpose.

The **Downtime Analysis Dashboard** provided an overview of key production metrics such as total production time, total downtime, downtime impact, and batch performance, supported by visual comparisons across operators, products, and shifts. Meanwhile, the **Downtime Causes Dashboard** focused on identifying and ranking the most significant downtime contributors using a Pareto approach, as well as visualizing downtime distribution across products, operators, shifts, and days.

Together, these dashboards delivered a comprehensive and intuitive view of manufacturing performance, allowing decision-makers to easily explore the data, uncover inefficiencies, and pinpoint opportunities for reducing downtime and improving productivity.

7 Recommendations